격자 안에서의 완전탐색

완전탐색이란, **문제를 해결할 수 있는 가장 naive한 방법**입니다. 이는 모든 가능한 경우를 다 따져보는 것으로, 코드를 작성하기가 쉽다는 장점이 있지만 모든 가능한 경우를 전부 계산해봐야 하므로 올바른 정답을 찾기까지 걸리는 시간(프로그램 수행시간)이 비교적 오래 걸립니다.

따라서 완전탐색은 시간복잡도를 계산해보고, 시간초과가 나지 않을 것 같다면 항상 적용해야 하는 방법이라 할 수 있습니다. **반복문을 활용한 완전탐색**에 대해 아직 잘 모르신다면, Novice Mid 문제집의 완전탐색 I, 완전탐색 II, 완전탐색 III 단원을 공부하시고 나서 진행하는 것을 추천드립니다.

완전탐색은 예는 크게 2가지를 들어볼 수 있습니다.

1. **4자리 자전거 자물쇠**를 풀기 위한 가장 쉬운 방법

이 경우에는 0000부터 9999까지 일일이 다 맞춰보면 됩니다.

2. **N개의 숫자 중 M개의 숫자**를 적절히 골라 합이 최대가 되도록 하기
이 경우에는 직접 가능한 모든 조합 C(N, M)개를 만들어 보며 그 합을 구해, 그 중 최댓값을 구하면 됩니다.

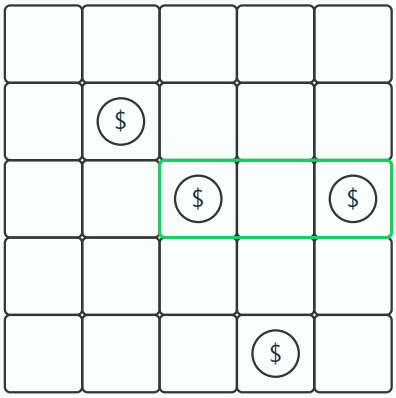
완전탐색을 구현할 수 있는 방법은 **For문** 기반으로 구현하는 것과, 재귀함수 기반의 **Backtracking(Brute Force)** 기법을 이용하는 방법, 이렇게 크게 2가지 입니다. 그 중에서도 이번에는 For문을 기반으로 하며, 동시에 **격자 위에서** 진행되는 시뮬레이션에 대해 알아보도록 하겠습니다.

다음 문제를 한번 살펴보겠습니다.

> n * n 크기의 격자에 1 * 3 크기의 직사각형을 잘 잡았을 때,
> 최대로 얻을 수 있는 돈을 얼마인지 구해보세요.

이때 편의상 돈이 있는 곳은 1, 없는 곳은 0으로 나타낸다고 생각해봅시다.
그렇다면 아래 예시에서는 답이 2가 됩니다.

codetree ▲

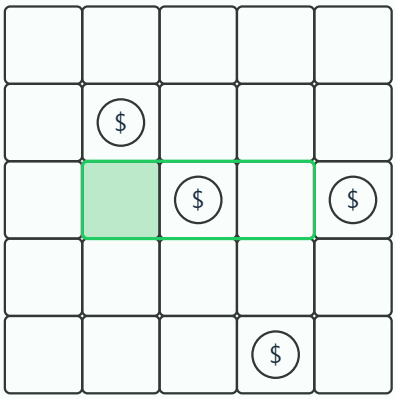


이 문제에서는 어느 지점을 잡아야 최대의 돈을 얻을 수 있을지 바로 판단하기가 어렵습니다.
따라서 가능한 모든 지점에 1 * 3 크기의 직사각형을 놓아보며, 얻을 수 있는 돈을 세고, 그 중 최댓값을 구하면 됩니다.

이 과정은 크게 두 부분으로 나눠 진행해볼 수 있습니다.

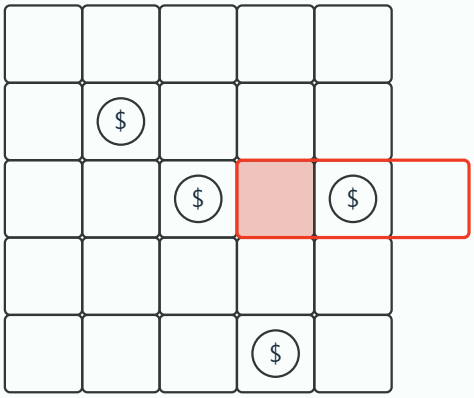
• Step 1. 모든 가능한 1 * 3 격자의 좌측 모서리를 잡아봅니다.
1 * 3 크기의 직사각형을 놓을 지점을 좌측 모서리를 기준으로 생각해볼 수 있습니다.

codetree ▲



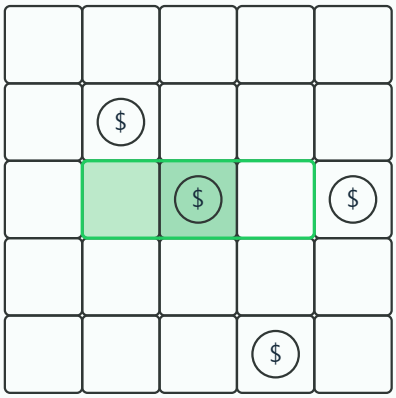
다만 이때 유의해야할 점은, 1 * 3 직사각형이 격자를 벗어나는 경우는 꼭 제외해야 한다는 것입니다.

codetree ▲



• Step 2. 좌측 모서리가 잡혔다면, 1 * 3 칸 내 동전의 수를 세어 최댓값을 갱신해줍니다.

codetree ▲



따라서 격자에서의 완전탐색은 칸 단위로 원하는 지점을 모두 잡아 확인하는 식으로 진행하면 됩니다.

Disclaimer : Copyright © Branch & Bound
코드트리의 모든 교육 자료는 저작권법으로 보호됩니다.
© Branch & Bound의 동의 없이 무단으로 재생산/복제/배포할 수 없습니다.